



考点	重要程度	占分	题型
1. 霍夫曼编码	★★★★	10-15	大题
2. LZW编码	★★★★	10-15	大题

一、 图像编码

一、霍夫曼编码

1.霍夫曼编码的步骤

霍夫曼编码使用变长编码表对源符号（如文件中的一个字母）进行编码，出现机率高的字母使用较短的编码，反之出现机率低的则使用较长的编码，这便使编码之后的字符串的平均长度降低，从而达到无损压缩数据的目的。

思想：出现概率大的数据用短码表示，出现概率小的数据用长码表示。



扫码观看
视频讲解更清晰

一、霍夫曼编码

霍夫曼编码的具体步骤如下：

1. 将信源符号的概率按减小的顺序排列。
2. 把两个最小的概率相加，并继续这一步骤，始终将较高的概率分支放在右边，直到最后变成概率 1。
3. 画出由概率 1 处到每个信源符号的路径，顺序记下沿路径的 0 和 1，所得就是该符号的霍夫曼码字。
4. 将每对组合的左边一个指定为 0，右边一个指定为 1（或相反）。

【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

解： 1. 首先计算出每个字符出现的次数（概率）：

字符	次数
B	10
A	8
C	3
D	4
E	5

2. 把出现次数（概率）最小的两个相加，并作为左右子树，重复此过程，直到概率值为1

B:10

A:8

C:3

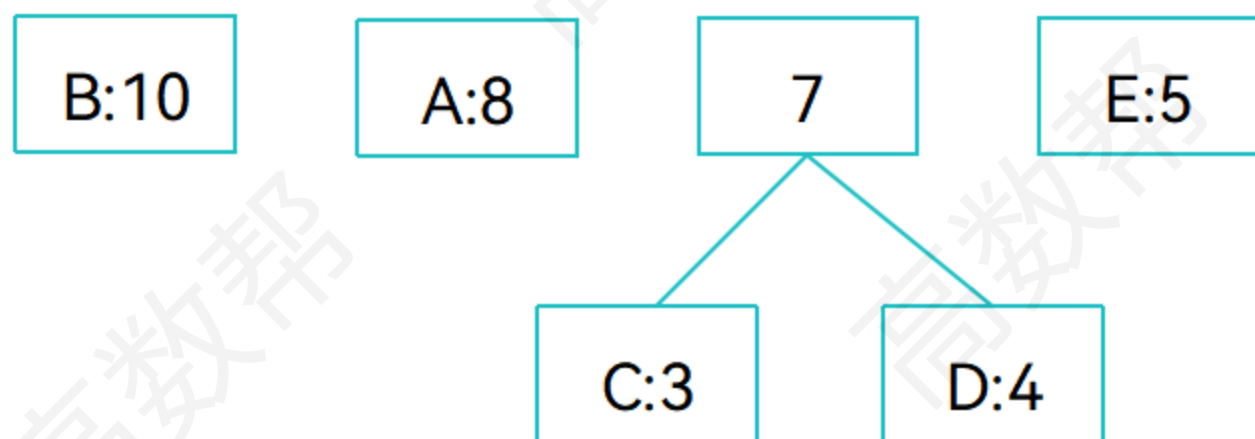
D:4

E:5

【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

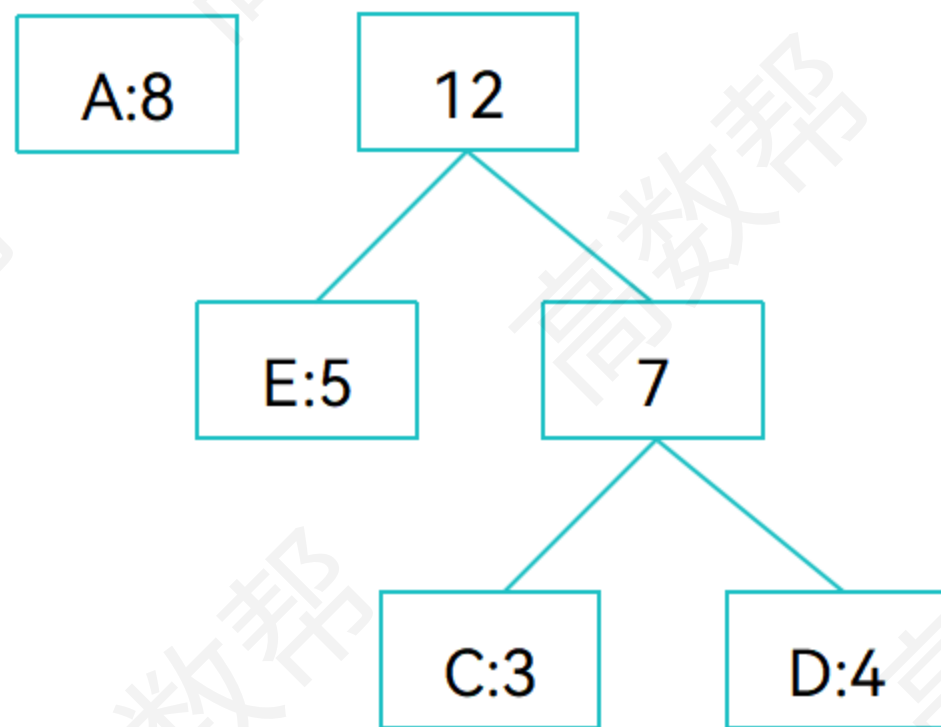
第一次：将概率最低值3和4相加，组合成7；



【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

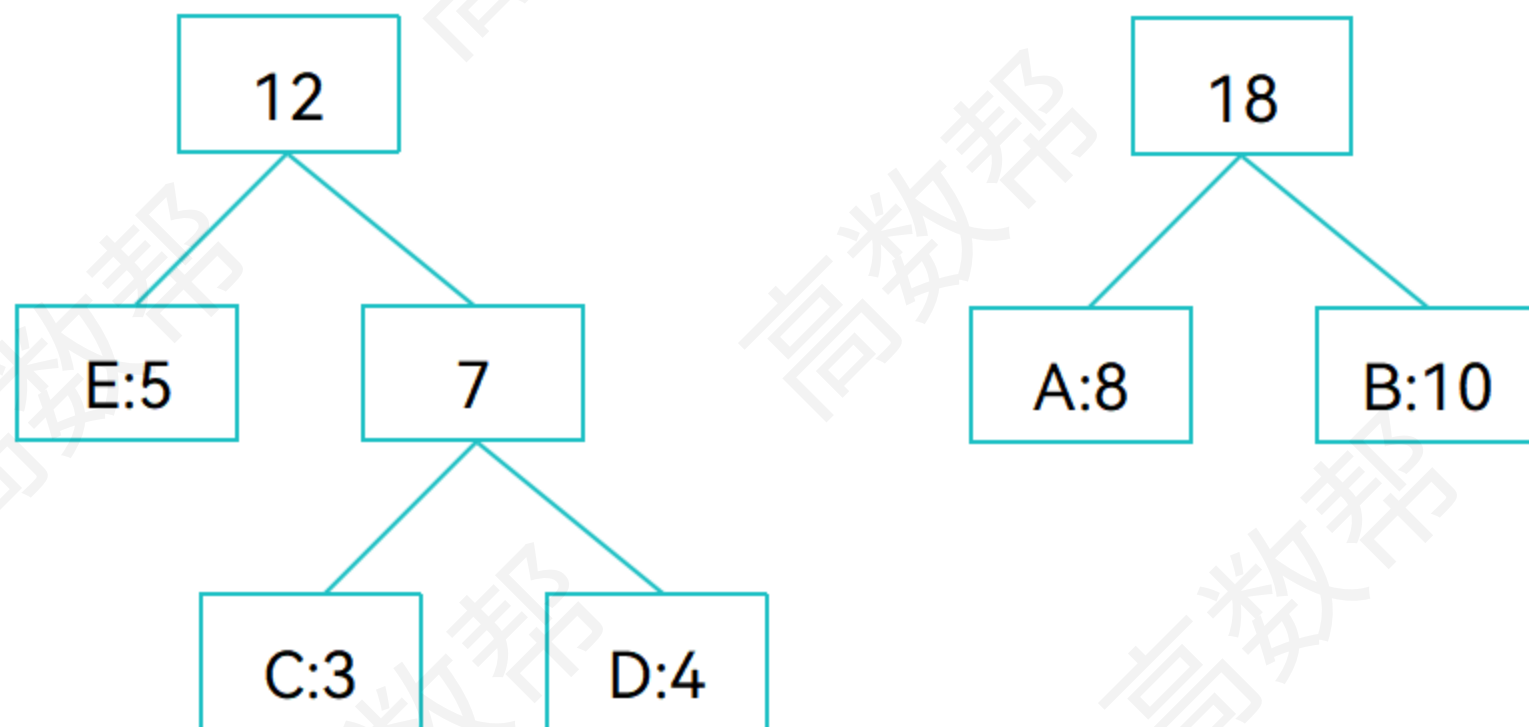
第二次：将最低值5和7相加，组合成12：



【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

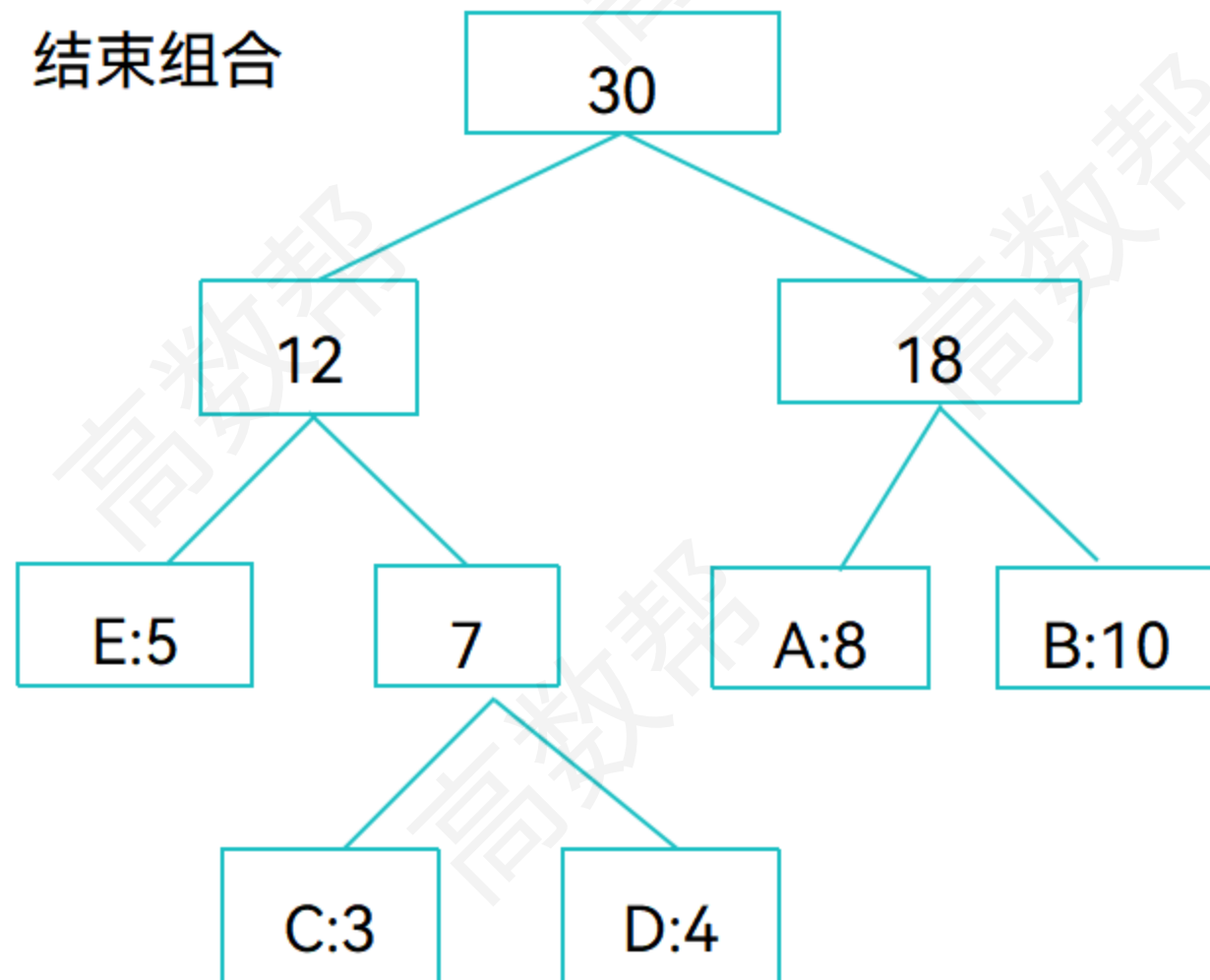
第三次：将8和10相加，组合成18；



【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

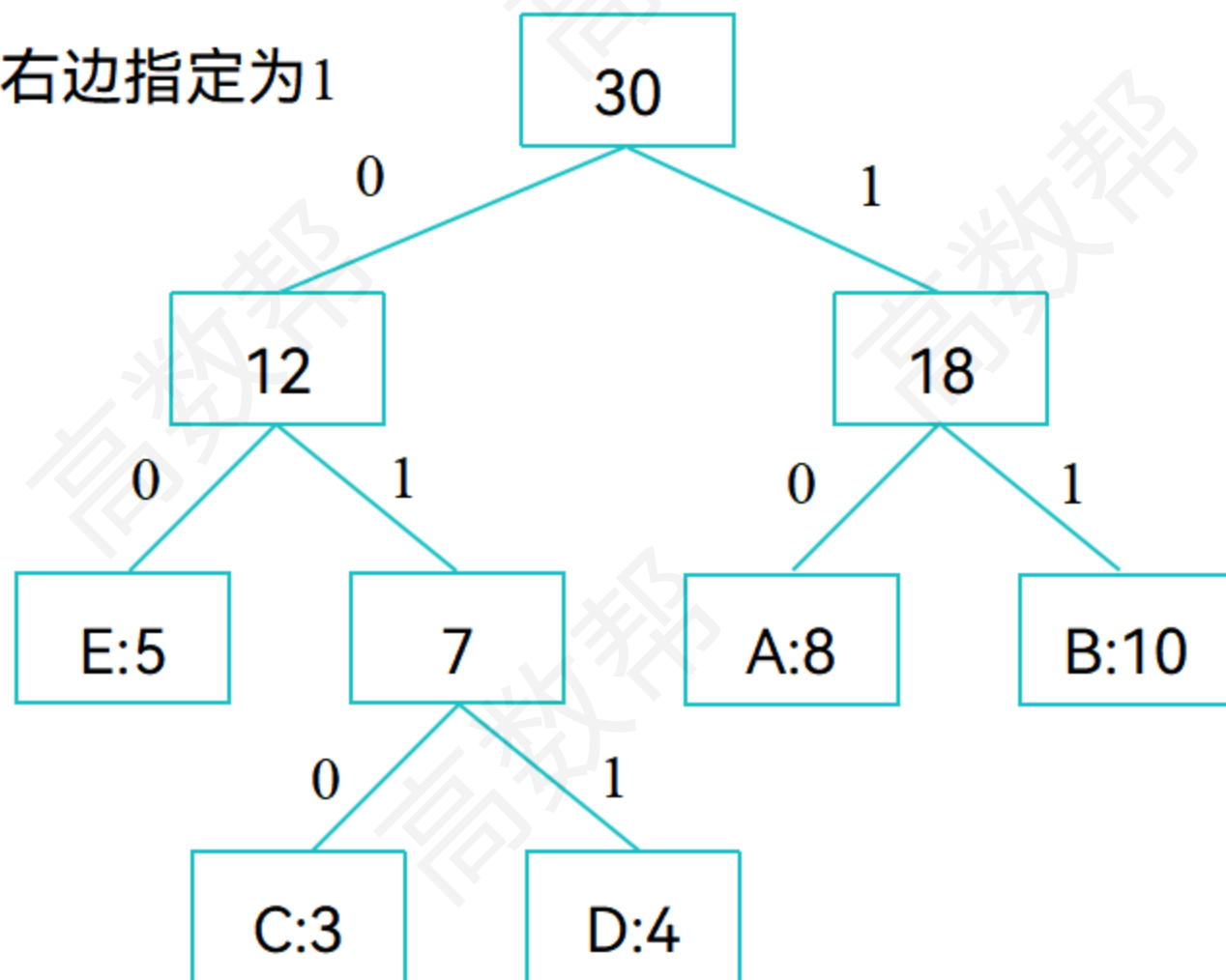
第四次：将最低值12和18相加，结束组合



【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

3. 将每个二叉树的左边指定为0，右边指定为1



【题1】 现有一个由5个不同符号组成的30个符号的字符串：

BABACAC ADADABB CBABEBE DDABEEEB

4.沿二叉树顶部到每个字符路径，获得每个符号的编码

字符	次数	编码
B	10	11
A	8	10
C	3	010
D	4	011
E	5	00

一、霍夫曼编码

概率大，编码也越短，概率小，编码也越长。

若收到“10111101100”那么其解码后就是“ABBDE”。所以，我们需要通过
这个二叉树建立我们Huffman编码和解码的字典表。

Huffman编码使得每一个字符的编码都与另一个字符编码的前一部分不同，
不会出现像'A': 00, 'B': 001, 这样的情况，解码也不会出现冲突。

一、霍夫曼编码

2.霍夫曼编码的局限性

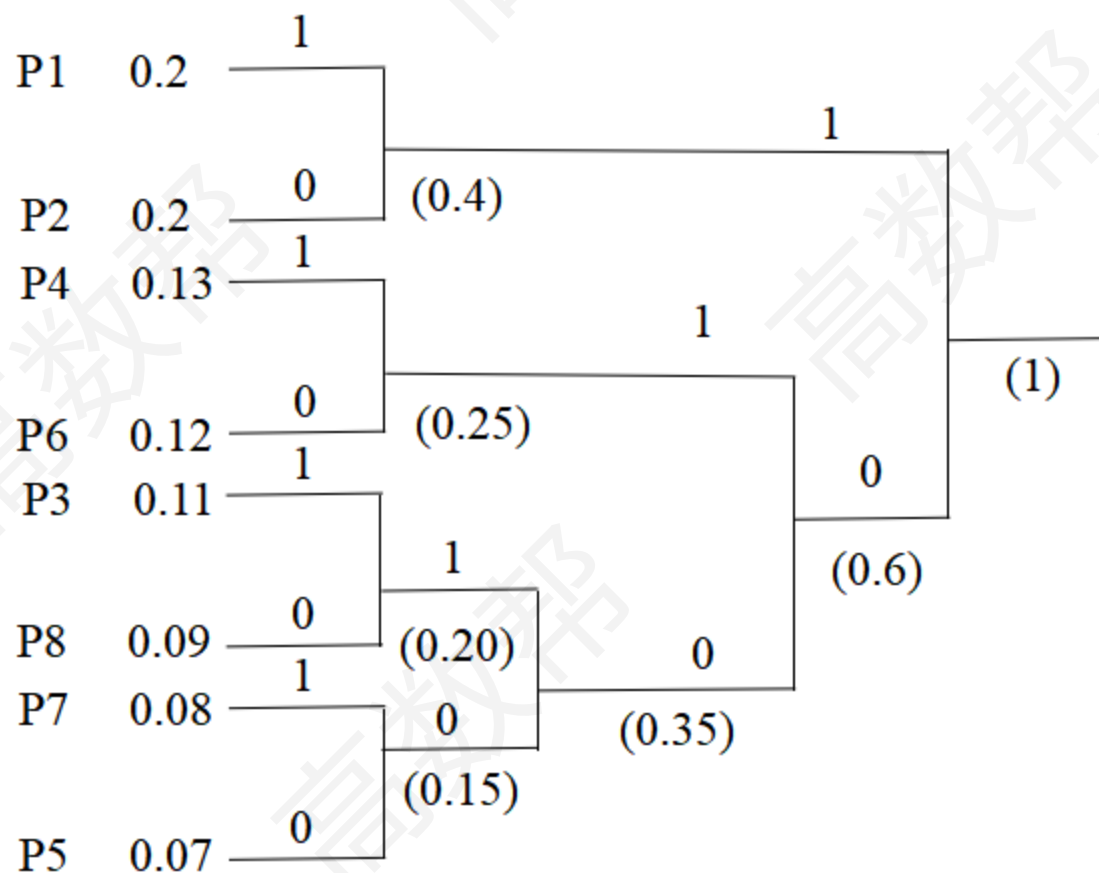
1. 源符号集的概率分布不是 2^{-n} 次方的形式，则无法达到熵极限；
2. 码复杂；需要实现知道输入符号集的概率分布；没有错误保护功能。



扫码观看
视频讲解更清晰

【题2】 有一信源X，其中： $P_1=0.20$, $P_2=0.20$, $P_3=0.11$, $P_4=0.13$, $P_5=0.07$,
 $P_6=0.12$, $P_7=0.08$, $P_8=0.09$, 将其进行霍夫曼编码，并计算信源的熵、平均码长、编码效率及冗余度，若采用二叉树编码，请画出二叉树。

解： 1. 编码的时候从右往左开始编码。



P1: 11
 P2: 10
 P4: 011
 P6: 010
 P3: 0011
 P8: 0010
 P7: 0001
 P5: 0000

【题2】 有一信源X，其中： $P_1=0.20$, $P_2=0.20$, $P_3=0.11$, $P_4=0.13$, $P_5=0.07$,
 $P_6=0.12$, $P_7=0.08$, $P_8=0.09$,将其进行霍夫曼编码，并计算信源的熵、平均码长、编码效率及冗余度，若采用二叉树编码，请画出二叉树。

2.用二叉树进行霍夫曼编码算法步骤如下：

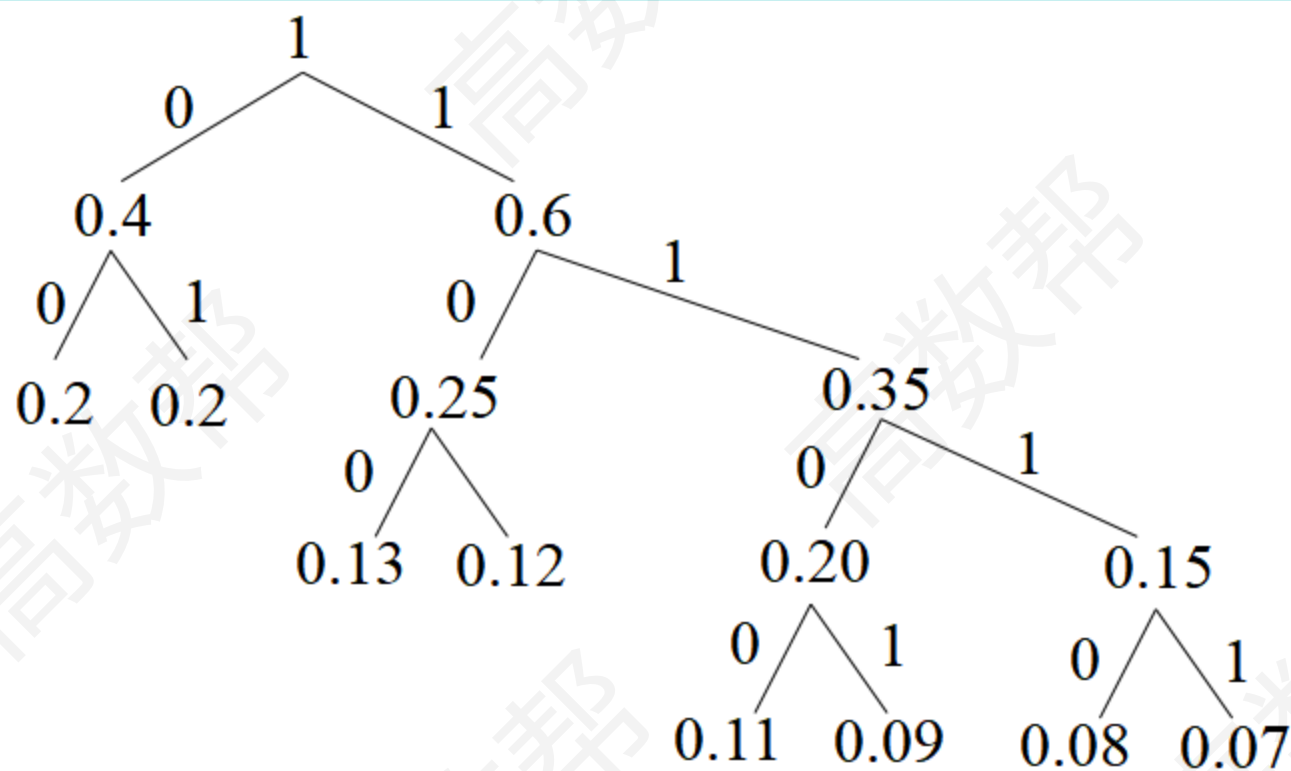
Step1.统计每个元素出现的频率，且从左到右按照从大到小排列；

Step2.选出频率最小的两个值,作为二叉树的两个叶子节点,将其和作为它们的根节点,两个叶子节点不再参与排序,新的根节点同其余元素出现的频率排序。

Step3.重复Step2,直到最后得到和为1的根节点。

Step4.将形成的二叉树的子节点左边标0,右边标1。把最上面的根节点到最下面的叶子节点途中遇到的0,1序列串起来,就得到了各个元素的编码。

【题2】 有一信源X，其中： $P_1=0.20$, $P_2=0.20$, $P_3=0.11$, $P_4=0.13$, $P_5=0.07$,
 $P_6=0.12$, $P_7=0.08$, $P_8=0.09$,将其进行霍夫曼编码，并计算信源的熵、平均码长、编码效率及冗余度，若采用二叉树编码，请画出二叉树。



P1:	0.2	00
P2:	0.2	01
P4:	0.13	100
P6:	0.12	101
P3:	0.11	1100
P8:	0.09	1101
P7:	0.08	1110
P5:	0.07	1111

【题2】 有一信源X，其中： $P_1=0.20$, $P_2=0.20$, $P_3=0.11$, $P_4=0.13$, $P_5=0.07$,
 $P_6=0.12$, $P_7=0.08$, $P_8=0.09$,将其进行霍夫曼编码，并计算信源的熵、平均码长、编码效率及冗余度，若采用二叉树编码，请画出二叉树。

信源熵： $H(S) = - (2 \times 0.2 \log_2 0.2 + 0.13 \log_2 0.13 + 0.12 \log_2 0.12 + 0.11 \log_2 0.11) \\ + 0.09 \log_2 0.09 + 0.08 \log_2 0.08 + 0.07 \log_2 0.07) = 0.90$

平均码长： $\bar{L} : 0.2 \times 2 \times 2 + 0.13 \times 3 + 0.12 \times 3 + 0.11 \times 4 + 0.09 \times 4 + 0.08 \times 4 + 0.07 \times 4 = 2.95$

编码效率： $\eta = \frac{H(S)}{\bar{L}} = \frac{0.9}{2.95} \times 100\% = 98.3\%$

冗余度： $r = 1 - \eta = 1.7\%$

二、LZW编码

1. LZW编码

LZW编码就是把出现过的字符串映射到记号上，这样就可能用较短的编码来表示长的字符串，实现压缩，例如对于字符串：ABABAB

可以看到子串AB在后面重复出现了，这样我们可以用一个特殊记号表示AB，例如数字2，这样原来的字符串就可以表示为：AB22

这里我们称2是字符串AB的记号。A和B也可以有记号，例如规定数字0表示A，数字1表示B。实际上最后得到的压缩后的数据应该是一个记号流。这样我们就有一个记号和字符串的映射表，即字典。

二、LZW编码

1. LZW编码

Symbol	String
0	A
1	B
2	AB

有了压缩后的编码0122，结合字典，就能够很轻松地解码)原字符串ABABAB。



扫码观看
视频讲解更清晰

二、LZW编码

3. 编码算法

编码器从原字符串不断地读入新的字符，并试图将单个字符或字符串编码为记号。这里我们维护两个变量，一个是P (Previous)，表示手头已有的，还没有被编码的字符串，一个是C (current)，表示当前新读进来的字符。

1. 初始状态，字典里只有所有的默认项，如0->a, 1->b, 2->c。此时P和C都是空的。
2. 读入新的字符C，与P合并形成字符串P+C。（收尾的时候有一些特殊的处理，如果到达字符串尾部，没有新的C读入了，则将手头的P对应的记号输出，结束。）
3. 在字典里查找P+C，如果：
 - P+C在字典里， $P=P+C$ ；
 - P+C不在字典里，将P的记号输出；在字典中为P+C建立一个记号映射；更新 $P=C$ 。
4. 返回步骤2重复，直至读完原字符串中所有字符。

二、LZW编码

3. 编码算法

P是可以被编码为记号的子串。但还并未输出。新的字符C不断被读入并添加到P的尾部，只要P+C仍然能在字典里找到，就不断增长更新 $P=P+C$ 。当新的P+C无法在字典里找到时，则输出已有的P的编码记号，并为新子串P+C建立字典表项。P则更新为单字符C，重新开始增长。



扫码观看
视频讲解更清晰

【题3】 对ababcbababac进行LZW编码。

初始状态字典里有三个默认的映射

Symbol	String
0	a
1	b
2	c

解： 开始编码：

Step	P	C	P+C	P+C in Dict	Action	Output
1	-	a	a	Yes	更新P=a	-
2	a	b	ab	No	添加3->ab,更新P=b	0
3	b	a	ba	No	添加4->ba,更新P=a	1
4	a	b	ab	Yes	更新P=ab	-
5	ab	c	abc	No	添加5->abc,更新P=c	3
6	c	a	ca	No	添加6->ca,更新P=a	2
7	a	b	ab	Yes	更新P=ab	-
8	ab	a	aba	No	添加7->aba,更新P=a	3
9	a	b	ab	Yes	更新P=ab	-
10	ab	a	aba	Yes	更新P=aba	-
11	aba	c	abac	No	添加8->abac,更新P=c	7
12	c	-	-	-		2

【题3】 对ababcbababac进行LZW编码。

初始状态字典里有三个默认的映射

Symbol	String
0	a
1	b
2	c

输出的结果为0132372，完整的字典为：

Symbol	String
0	a
1	b
2	c
3	ab
4	ba
5	abc
6	ca
7	aba
8	abac

二、LZW编码

4. 解码

解码器的输入是压缩后的数据，类似于编码，我们仍然维护两个变量 pW 和 cW ， pW 表示刚刚解码的记号； cW 表示当前新读进来的记号。

用 $Str(cW)$ 和 $Str(pW)$ 表示它们解码出来的原字符串。

1. 初始状态，字典里只有所有的默认项，例如 $0 \rightarrow a$ ， $1 \rightarrow b$ ， $2 \rightarrow c$ 。此时 pW 和 cW 都是空的。
2. 读入第一个的符号 cW ，解码输出。
3. 赋值 $pW = cW$ 。
4. 读入下一个符号 cW 。

二、LZW编码

4. 解码

5. 在字典里查找 cW ，如果：

a. cW 在字典里：

(1) 解码 cW ，即输出 $\text{Str}(cW)$ 。

(2) 令 $P=\text{Str}(pW)$ 。 $C=\text{Str}(cW)$ 的**第一个字符**。

(3) 在字典中为 $P+C$ 添加新的记号映射。

b. cW 不在字典里：

(1) 令 $P=\text{Str}(pW)$ 。 $C=\text{Str}(pW)$ 的**第一个字符**。

(2) 在字典中为 $P+C$ 添加新的记号映射，这个新的记号一定就是 cW 。

(3) 输出 $P+C$ 。

二、LZW编码

4. 解码

6. 返回步骤3重复，直至读完所有记号。收到的结果进行解码：0 1 3 2 3 7 2。

手里的字典只有默认项：

Symbol	String
0	a
1	b
2	c

二、LZW编码

4. 解码

解码:

Step	pW	cW	cW in Dict	Action	Output
1	0	1	Yes	$P=a, C=b, P+C=ab$, 添加3- $\rightarrow ab$	b
2	1	3	Yes	$P=b, C=a, P+C=ba$, 添加4- $\rightarrow ba$	ab
3	3	2	Yes	$P=ab, C=c, P+C=abc$, 添加5- $\rightarrow abc$	c
4	2	3	Yes	$P=c, C=a, P+C=ca$, 添加6- $\rightarrow ca$	ab
5	3	7	No	$P=ab, C=a, P+C=aba$, 添加7- $\rightarrow aba$	aba
6	7	2	Yes	$P=aba, C=c, P+C=abac$, 添加8- $\rightarrow abac$	c

【题4】 一图像灰度值分布见表。假设有一个有0-255灰度值一一映射的字典，位置256到511开始时未使用。图像按从左到右、从上到下处理像素的方法来编码，十六个像素的编码过程如右表所示：

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

解： 开始编码：

Step	当前可识别序列P	当前字符C	P+C	P+C是否在字典	执行	输出
1		39	39	Y	$P=P+C$	
2	39	39	39-39	N	添加256->39-39, $P=C$	39
3	39	126	39-126	N	添加257->39-126, $P=C$	39
4	126	126	126-126	N	添加258->126-126, $P=C$	126
5	126	39	126-39	N	添加259->126-39, $P=C$	126
6	39	39	39-39	Y	$P=P+C$	
7	39-39	126	39-39-126	N	添加260->39-39-126, $P=C$	256
8	126	126	126-126	Y	$P=P+C$	
9	126-126	39	126-126-39	N	添加261->126-126-39, $P=C$	258
10	39	39	39-39	Y	$P=P+C$	
11	39-39	126	39-39-126	Y	$P=P+C$	
12	39-39-126	126	39-39-126-126	N	添加262->39-39-126-126, $P=C$	260
13	126	39	126-39	Y	$P=P+C$	
14	126-39	39	126-39-39	N	添加263->126-39-39, $P=C$	259
15	39	126	39-126	Y	$P=P+C$	
16	39-126	126	39-126-126	N	添加264->39-126-126, $P=C$	257
17	126					126

【题4】 一图像灰度值分布见表。假设有一个有0-255灰度值一一映射的字典，位置256到511开始时未使用。图像按从左到右、从上到下处理像素的方法来编码，十六个像素的编码过程如右表所示：

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

也就是说上面的十六个像素39 39 126 126 39 39 126 126 39 39 126 126 39 39 126 126被编码为39 39 126 126 256 258 260 259 257 126,最终字典新增加了9条规则。

解码如下：编码序列39 39 126 126 256 258 260 259 257 126，字典只有原始的没有更新规则前的，即大小为512字带有0-255灰度值一一映射的字典，位置256到511未使用。

【题4】一图像灰度值分布见表。假设有一个有0-255灰度值一一映射的字典，位置256到511开始时未使用。图像按从左到右、从上到下处理像素的方法来编码，十六个像素的编码过程如右表所示：

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

解码步骤如图：

Step	P	C	P+C in Dict	Action	Output
0		39	直接输出	$P=C=39$	-
1	39	39	Y	添加256- $\rightarrow dP+dC$ (第一个字)39-39, $P=C$	39
2	39	126	Y	添加257- $\rightarrow dP+dC$ (第一个字)39-126, $P=C$	39
3	126	126	Y	添加258- $\rightarrow dP+dC$ (第一个字)126-126, $P=C$	126
4	126	256	Y	添加259- $\rightarrow dP+dC$ (第一个字)126-39, $P=C$	126
5	256	258	Y	添加260- $\rightarrow dP+dC$ (第一个字)39-39-126, $P=C$	39-39
6	258	260	Y	添加261- $\rightarrow dP+dC$ (第一个字)126-126-39, $P=C$	126-126
7	260	259	Y	添加262- $\rightarrow dP+dC$ (第一个字)39-39-126-126, $P=C$	39-39-126
8	259	257	Y	添加263- $\rightarrow dP+dC$ (第一个字)126-39-39, $P=C$	126-39
9	257	126	Y	添加264- $\rightarrow dP+dC$ (第一个字)39-126-126, $P=C$	39-126
10	126	Null			126